阶段三：协同仿真与性能分析

位置 ③ · 官方 3.1.1、3.1.2、3.2、3.3

章节与图表沿用总目录第 1 ~ 10 章编号。正文给出技术方法、核心代码和实测结果；文末一份附录集中列出复现环境、产物哈希与工程材料路径。

目录

- 第 8 章　NEMU扩展与DiffTest协同验证
- 第 9 章　标准RVV等价基线设计与验证
- 第 10 章　分层性能测量与分析
- 附录　第三阶段复现环境、产物哈希与材料路径索引

NEMU扩展与DiffTest协同验证

本文要点

- 在源码 NEMU 中实现 `vdot.vv` 的译码、八项有符号点积、完整目的更新和异常语义，对应官方要求 3.1.1。
- 参考库 API 与 RTL/NEMU 整核协同验证分层检查；基础、异常、重叠及流水套件均通过，对应官方要求 3.1.2。
- DiffTest 发现短 VL 异常时 DUT 的 `mtval` 未携带指令字；修复 ROB 到 CSR 的信息传递后，原异常套件回归通过。
- 参考模型、应用独立期望值和动态波形分别检查架构结果、预期语义与内部执行过程。

1 对象

验证对象是固定八项 signed INT8 点积指令 `vdot.vv`。合法非零 VL 下，前 8 项的有符号乘积求和写入目的低 32 位，高 96 位清零；合法 VL=0 保留完整旧目的。类型、权限、`vstart`、掩码及异常条件遵循项目 v0.1 语义合同。

整核采用单核 MinimalConfig、VLEN=128、`EMU_TRACE=1`。XiangShan 基版本为 `16ae9ddcda54fc9a2fddffad73174cf793ac7814`，叠加 VDot 接入与 `mtval` 修复；NEMU 基版本为 `a48a11f208ff81ad765591d83271d1b339fb3861`，叠加相应参考语义。实际修改随源码快照和补丁保存。

依据：语义合同[材料 1] · NEMU 源码快照[材料 2] · 运行身份

2 方法

2.1 NEMU 指令扩展（官方 3.1.1）

NEMU 将 `vdot.vv` 接入现有译码与向量状态管理机制。合法性检查先于 VL=0 分支，确保空操作也遵守类型与权限约束；计算前快照源和掩码，避免目的与源重叠导致读取改写后的值。

表 8-1　NEMU 修改对象与作用

文件（相对 NEMU 根）	作用
`src/isa/riscv64/instr/decode.c`	为 custom-0、funct6=0、funct3=0 增加精确译码入口
`src/isa/riscv64/include/isa-all-instr.h`	注册 VDot 执行 ID
`src/isa/riscv64/instr/rvv/vcompute.h`	包含新增语义实现
`src/isa/riscv64/instr/rvv/vdot.h`	检查合法性，快照操作数，完成乘加、目的更新与 VS Dirty 更新

译码入口先匹配新增的 custom-0 位型，并生成 vdot 执行 ID；随后一条规则继续处理原有标准 OP-V 指令。以下摘录 decode.c：

```
#ifdef CONFIG_RVV
  def_INSTR_IDTAB("000000 ? ????? ????? 000 ????? 00010 11", OP_V  , vdot);
  def_INSTR_IDTAB("??????? ????? ????? ??? ????? 10101 ??", OP_V  , OP_V);
#endif // CONFIG_RVV
```

该执行 ID 对应 def_EHelper(vdot)，调用链从机器码识别进入下面的语义实现。以下为保存的 vdot.h 原文摘录：

```
// CIE vdot.vv v0.1: fixed eight signed INT8 products, whole-register result.
// Included after vcompute_impl.h/vcommon.h. No generic RVV tail/mask merging.
def_EHelper(vdot) {
  require_vector(true); // host/guest VS and vill, before the vl=0 path
  if ((vtype->val >> 8) != 0 || vtype->vsew != 0 || vtype->vlmul != 0 ||
      vstart->val != 0 || (vl->val != 0 && (vl->val < 8 || vl->val > 16))) {
    longjmp_exception(EX_II);
  }
  const uint32_t word = s->isa.instr.val;
  const unsigned vd = (word >> 7) & 31;
  const unsigned vs1 = (word >> 15) & 31;
  const unsigned vs2 = (word >> 20) & 31;
  const unsigned vm = (word >> 25) & 1;
  if (vl->val != 0) {
    // Snapshot before any write, including vd=v0 and all source overlaps.
    const uint64_t a = cpu.vr[vs1]._64[0];
    const uint64_t b = cpu.vr[vs2]._64[0];
    const uint64_t mask = cpu.vr[0]._64[0];
    int32_t sum = 0;
    for (unsigned i = 0; i < 8; ++i) {
      const int ai = ((int)((a >> (8*i)) & 255) ^ 128) - 128;
      const int bi = ((int)((b >> (8*i)) & 255) ^ 128) - 128;
      if (vm || ((mask >> i) & 1)) sum += ai * bi;
    }
    cpu.vr[vd]._64[0] = (uint32_t)sum;
    cpu.vr[vd]._64[1] = 0;
  }
  vstart->val = 0;
  vp_set_dirty(); // includes legal vl=0 and guest VS when virtualized
}
```

源码与语义合同的对应关系如下：

1. require_vector(true) 与紧随其后的条件检查处理权限、vtype、vstart 和 VL。
2. a、b、mask 保存改写目的之前的值；循环只读取前 8 个字节，并按 vm 或对应掩码位选择乘积。
3. (uint32_t)sum 将有符号结果的低 32 位写入目的，另一个 64 位字清零，从而满足高 96 位为零的布局。
4. 合法 VL=0 跳过目的更新；成功路径清 vstart 并调用既有 vp_set_dirty()。

证据：译码源文件[材料 3]，语义源文件[材料 4]

2.2 构建与协同运行

RTL、NEMU 和测试应用独立构建。修改 RTL 后生成新的 `emu`；修改 NEMU 后生成动态参考库；只修改应用时生成新镜像，并使用已确定的 `emu`、参考库和 Flash。这样可以将验证结果关联到具体改动及其产物。

运行时通过 `--diff=.../riscv64-nemu-interpreter-so` 显式选择源码参考库，并通过 `-F evidence/vdot-build-80mKCD/boot-flash.bin` 统一启动内容。DiffTest 在退休、异常等架构同步点推进参考模型，比较导出的架构状态。应用另外核对独立期望值，以减少 DUT 与参考模型共享错误时的漏检。

DiffTest 通过 `regcpy` 交换架构状态。按本配置两侧源码的字段顺序核对，交换区为 **1400 字节**，其中 **32 个 128 位向量寄存器占 512 字节**，起始偏移为 800 字节。参考库 API 检查完整向量寄存器和相关 CSR，整核联合运行进一步检验实际状态交换与指令执行的一致性。

验收检查参考库路径、DiffTest 启用信息、应用 PASS、GOOD TRAP 和被测程序退出码。脚本与 `tee` 的退出码单独保存，区分被测程序结果与日志收集结果。

证据：构建与复现入口[材料 5] · 接口布局核对[材料 6]

3 结果

3.1 参考模型 API 检查

真实参考库的状态注入、单步执行及回读检查通过。该层直接检查完整 32 个向量寄存器和相关 CSR；使用的本地库身份与服务器整核参考库分开记录。

表 8-2　参考模型 API 检查结果

类别	数量	检查目标
`ok_write`	93	合法非零 VL 的完整目的和状态更新
`ok_no_write`	40	合法 VL=0 保留旧目的及约定状态
`illegal`	135	非法状态的异常及状态保持
通过合计	268	上述三类之和
配置不适用	10	固定配置无法动态关闭 RVV，单列且不计入通过

证据：API 原始结果[材料 7] · 本地与服务器库身份

3.2 RTL/NEMU 协同验证（官方 3.1.2）

修复后的 RTL 与源码 NEMU 在下列定向套件中保持架构状态一致。 本文统一定义套件名称及计数；其他章节按套件名称引用。

表 8-3　定向整核套件定义与结果

套件	用例数	主要检查	保存运行	结果
SMOKE	7	正数、正负极值、掩码、VL=0/VL=16、完整 128 位结果及状态	异常修复回归	通过
EXCEPTION	64	62 次预期陷阱和 2 个合法保护例；检查 cause、mepc、mtval 及状态	异常修复回归	通过
OVERLAP	60	15 种源、目的、掩码相等关系	异常修复回归	通过
PIPELINE-DEPS	48	依赖关系；12 个测试程序	流水回归	通过
PIPELINE-MIXED	32	不同向量操作混发；8 个测试程序	流水回归	通过
PIPELINE-CANCEL-CSR	40	32 个分支场景和 8 个精确异常场景	流水回归	通过
PIPELINE 合计	120	上述三项流水套件之和	流水回归	通过

Hello-meijie 与 VADD 在修复后同轮回归通过。两种标准 RVV 基线的等价验证及各计时应用也使用固定源码参考库，分别在第 9、10 章报告。计时应用的周期数来自 RTL，参考模型承担正确性检查。

证据：异常修复回归[材料 8] · 流水套件回归[材料 9] · 标准 RVV 等价验证 · 延迟与分层性能

3.3 DiffTest 定位案例：短 VL 异常的 mtval 修复

短 VL 异常时，VDot 执行期异常未将指令字送入 DUT 的 `mtval`；补齐 ROB 到 CSR 的信息传递后，EXCEPTION 套件通过。 定位过程保留了原异常镜像、参考库配置及比较字段。

旧 `emu` 在 `short-1-vm0` 用例中输出：

```
mtval different at pc = 0x0080000218, right=0x0020818b, wrong=0
```

表 8-4　失败读数及含义

字段	值	含义
故障指令 PC / `mepc`	`0x8000021c`	发生短 VL 异常的 `vdot.vv`
指令字 / 参考 `mtval`	`0x0020818b`	参考模型写入的异常指令字
DUT `mtval`	`0x00000000`	差分错误中的 `wrong` 值
差分显示 PC	`0x80000218`	前一条比较指令的位置，与故障 PC 分列

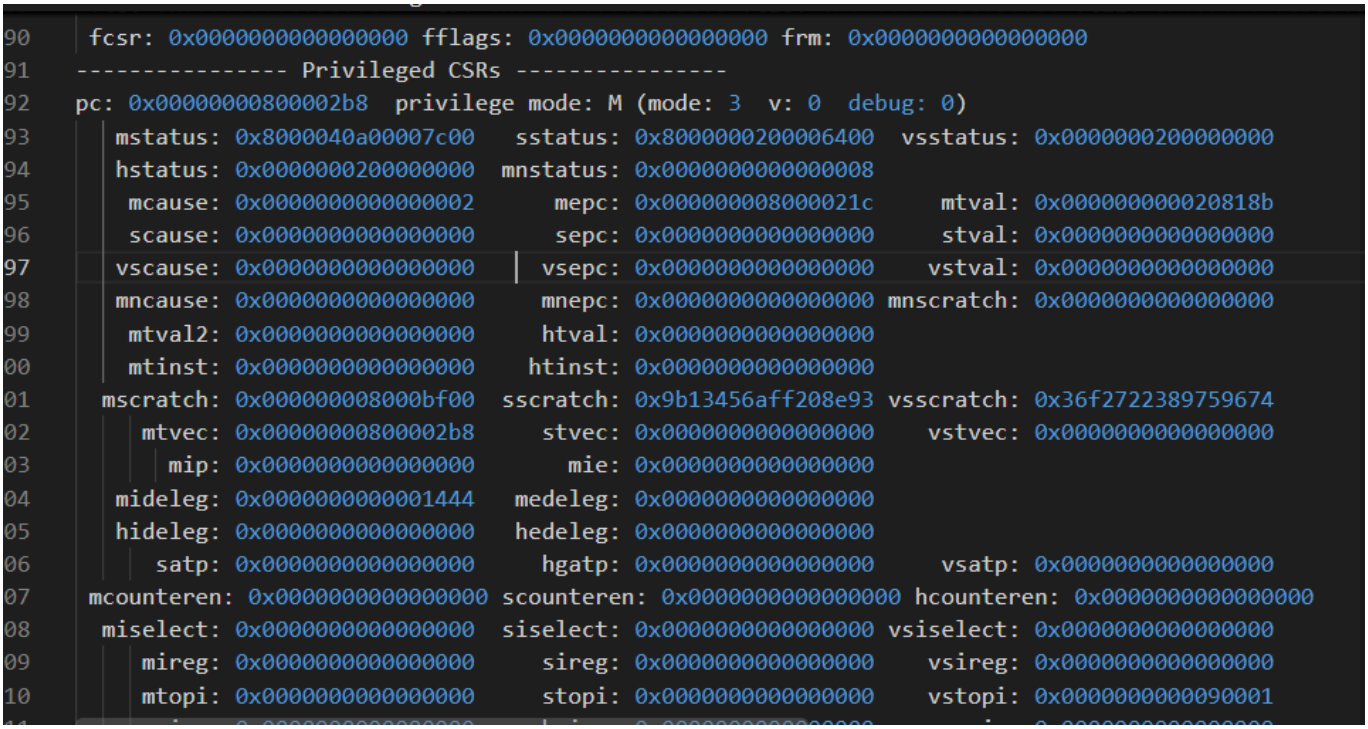


图 8-1　参考模型的异常快照显示 mcause=2、mepc=0x8000021c、mtval=0x0020818b（原证据 S2-F07a）。

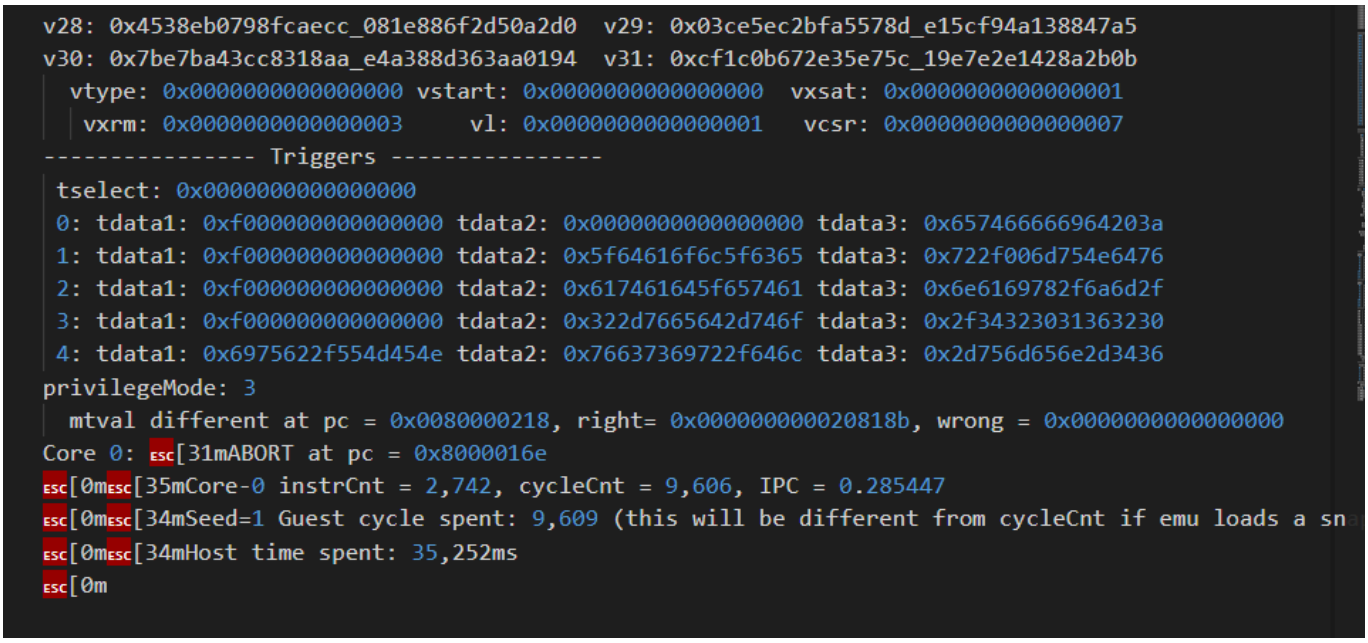


图 8-2　DiffTest 报告参考值 0x0020818b 与 DUT 值 0x00000000 不一致，随后 ABORT（原证据 S2-F07b）。

短 VL 由 VDot 执行阶段产生 `illegalInstr`，原 `trapInstInfo` 链主要携带译码期非法指令信息。修复在 ROB 功能性槽位保存指令编码及 generation，经精确异常选择后送入 CSR，使异常值与同一动态指令关联，并沿用原异常仲裁和 NEMU 语义。

修复生成新的 `emu`，使用原异常镜像和参考库配置回归。EXCEPTION、OVERLAP、SMOKE 及 Hello/VADD 均通过，旧 `emu` 与失败记录保留用于追溯。

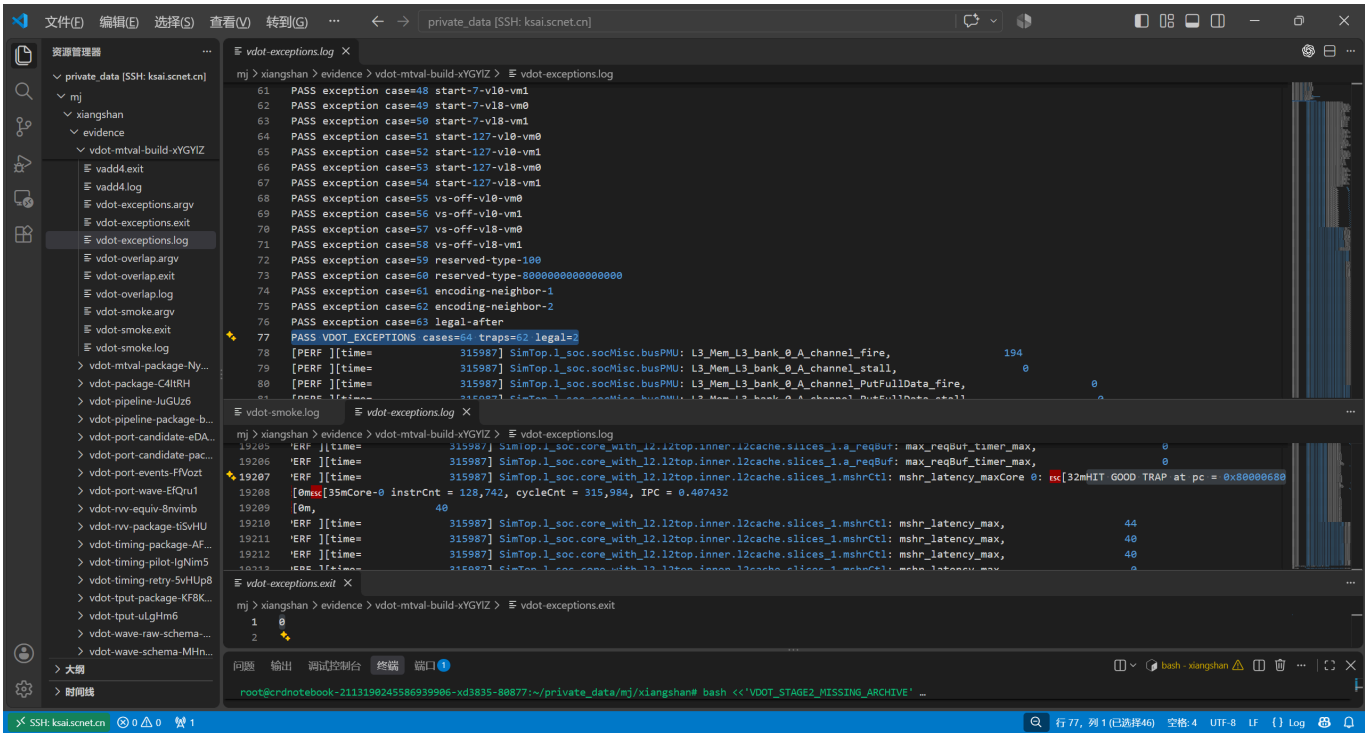


图 8-3　修复后 EXCEPTION 套件通过，GOOD TRAP 与退出码 0 同时可见（原证据 S2-F02a）。

证据：差分诊断与 ELF 映射[材料 10]．修复设计与补丁[材料 11]．修复后回归[材料 8]

3.4 重叠与流水回归记录

OVERLAP 和 PIPELINE 套件分别检查寄存器别名与依赖、混发、分支及精确异常情形。图 8-4～8-7 展示对应运行结果；内部取消、CSR 请求和端口竞争的动态实例在第 7 章用波形单独关联。

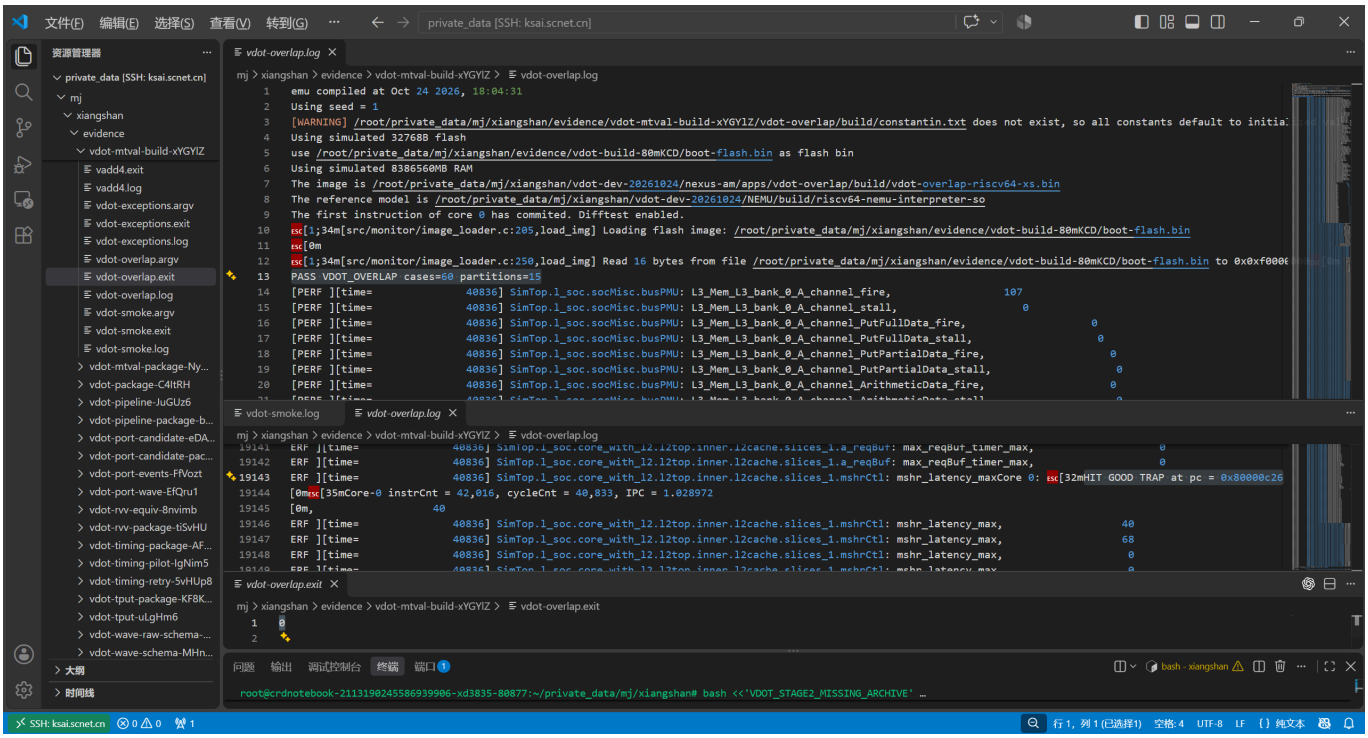


图 8-4　OVERLAP 套件通过，DiffTest、GOOD TRAP 与退出码 0 可见（原证据 S2-F02b）。

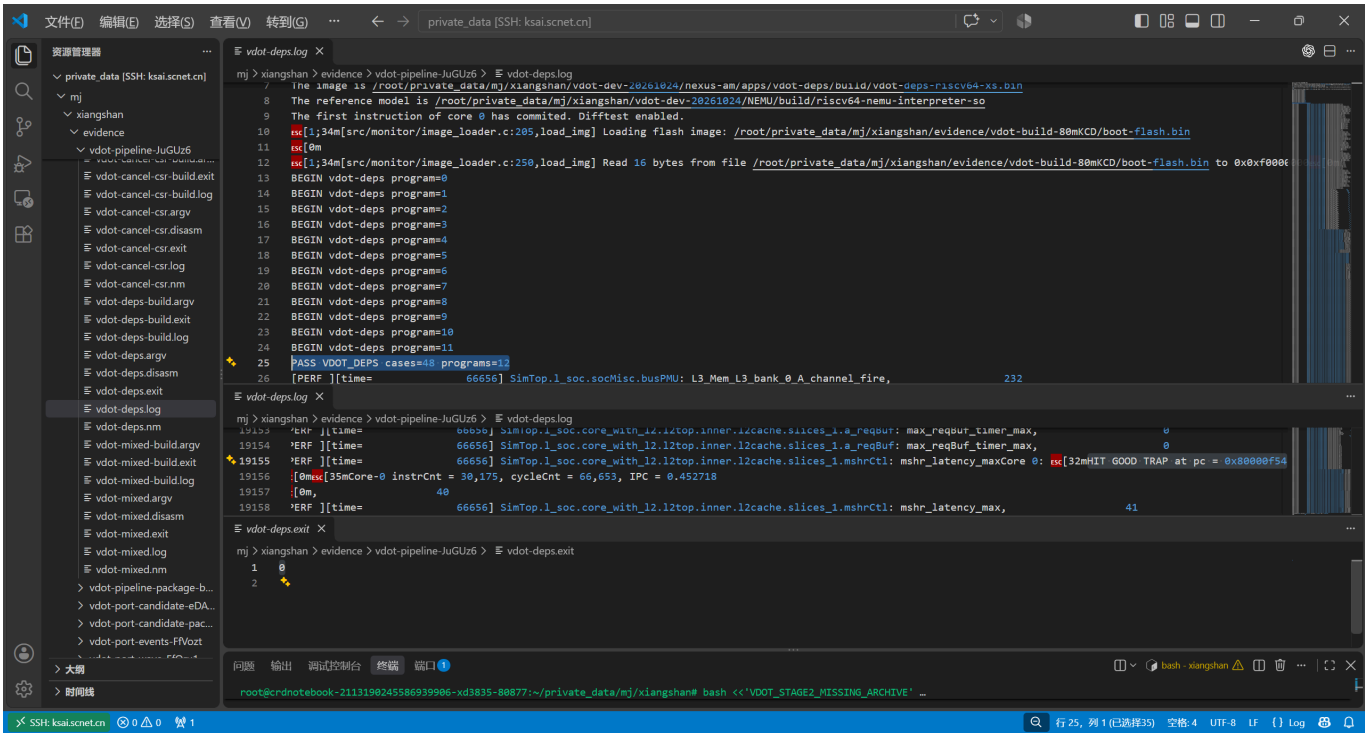


图 8-5　PIPELINE-DEPS 套件通过，日志给出依赖程序汇总与正常结束记录（原证据 S2-F03a）。

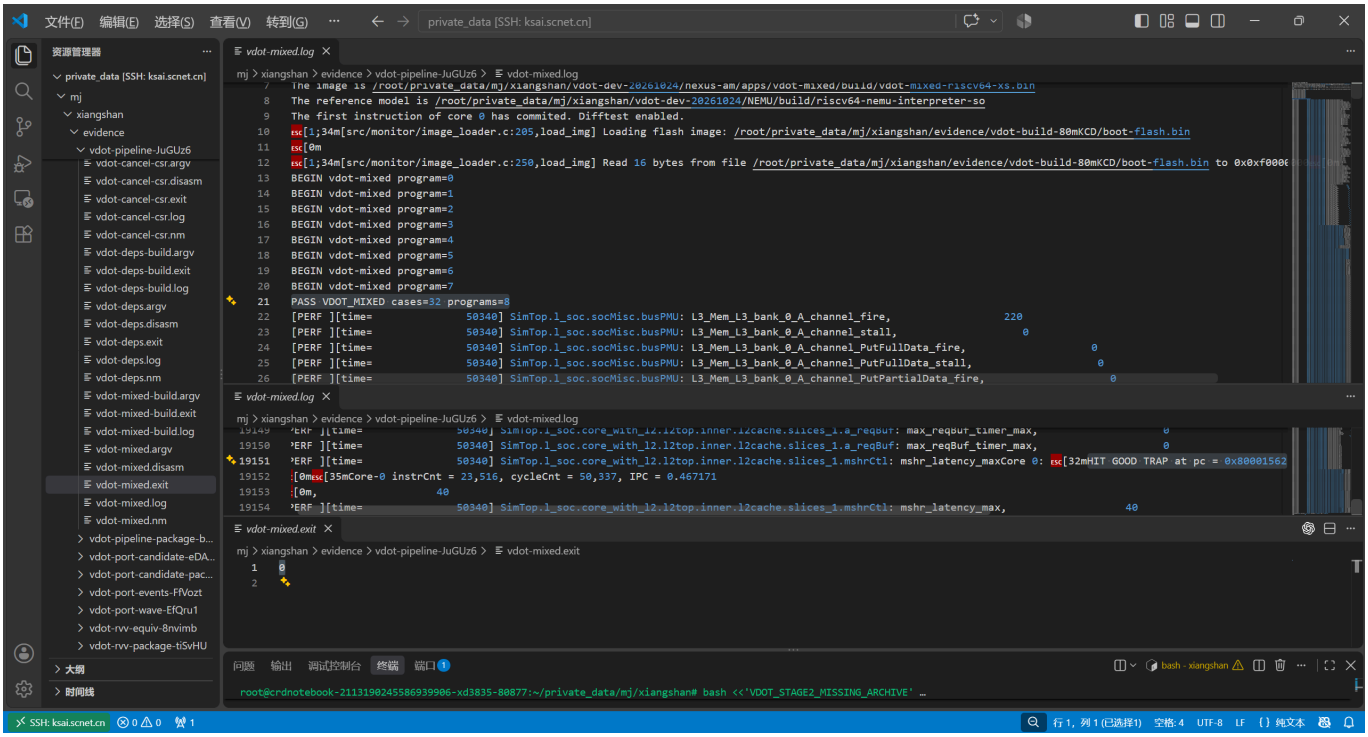


图 8-6　PIPELINE-MIXED 套件通过，记录选定混发场景的架构结果（原证据 S2-F03b）。

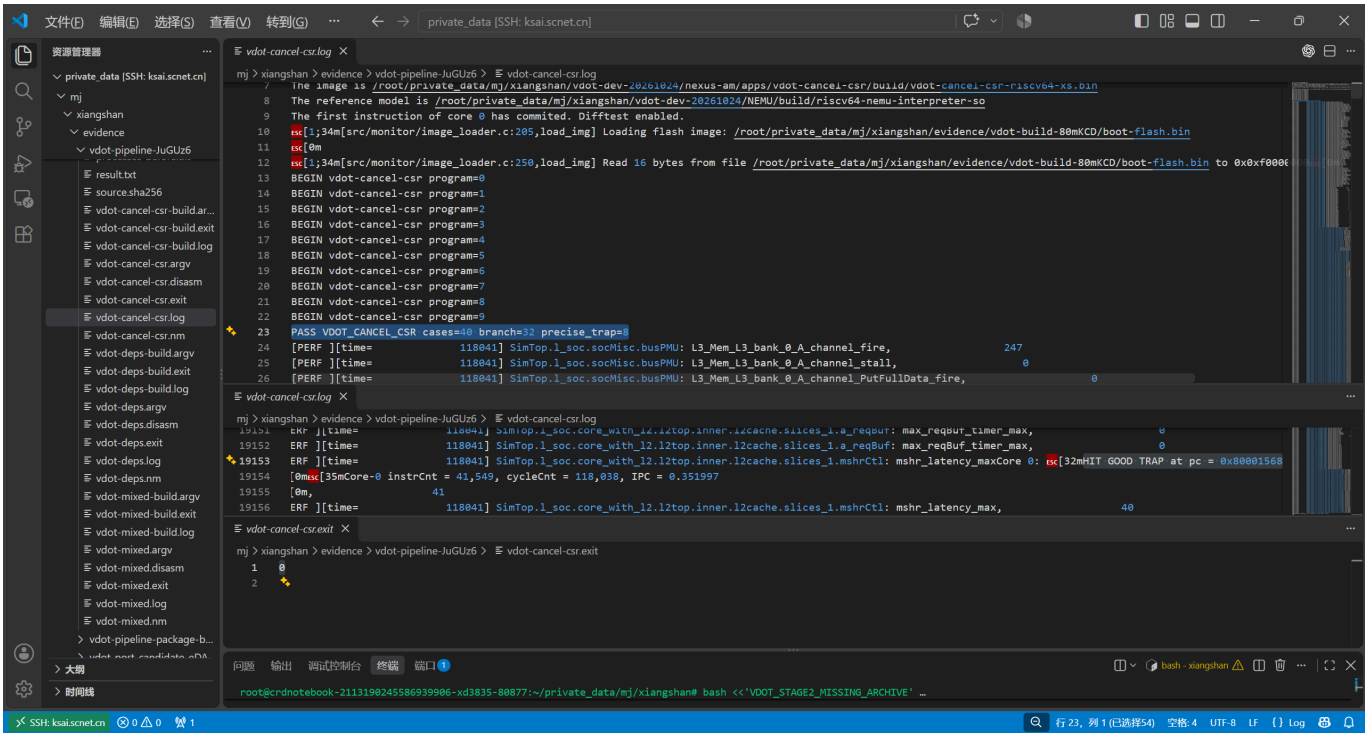


图 8-7　PIPELINE-CANCEL-CSR 套件通过，日志分别列出分支和精确异常场景（原证据 S2-F03c）。

证据：代表性微架构波形[材料 12]．原始日志与产物索引

4 总结与实验边界

NEMU 扩展与 RTL 协同验证形成了“语义实现—独立期望—差分定位—修复回归”的闭环。`mtval` 案例说明差分比较能够暴露执行结果之外的精确异常信息问题。

- 结果适用于表 8-2、表 8-3 的指定用例及固定配置。套件包含重复的功能检查，其数量不相加作为独立覆盖率。
- DiffTest 检查导出的架构状态；物理寄存器、取消时刻和仲裁过程由第 7 章的动态波形补充。已验证单例的取消窗口、CSR 接收 Dirty 请求和发射前端口竞争各有明确观测范围。
- A01 ~ A09 为项目语义合同的工程约定。第 2.2 节说明固定配置的状态交换布局，附录分别列出本地和服务器的构建身份。

标准RVV等价基线设计与验证

本文要点

- 使用标准加宽乘法与加宽归约构造 RVV-m1、RVV-mf2 两条基线，对应官方要求 3.2。
- 两条基线保持与 `vdot.vv` 相同的完整 128 位结果、VL=0 旧目的及约定 CSR 状态。
- 等价矩阵的所有路径均通过独立期望检查与服务器 RTL/NEMU 协同验证，检查器负向测试能拒绝错误结果。
- 基线内的配置、清零和恢复属于完整操作；其延迟加速比在第 10 章单独测量。

1 对象

本章验证标准 RVV 指令组合在指定调用约定下，能否得到与 `vdot.vv` 相同的完整结果和状态。入口采用合法 M 态、e8/m1、`vstart=0` 与可用向量状态；固定 `v0` 为掩码，`v1/v2` 为源，`v3` 为目的，预留 `v8/v9/v16` 和 `t0` 作为临时寄存器。

比较位置为汇编 `kernel_end`，位于测试壳改配置、存储结果之前。每条路径对照独立期望值，再进行路径间比较。此处建立等价关系，为后续 Baseline 与 VDot 的延迟比较提供相同语义前提。

2 方法

2.1 标准 RVV 组合实现（官方 3.2）

两条基线先以 `vwmul.vv` 保留 signed INT8 乘积的完整 INT16 位宽，再以 `vwredsum.vs` 归约为 INT32。归约使用独立零种子；预先清零完整目的 `v3`，并采用 `tu` 策略保留其高位零，得到自定义指令要求的输出布局。掩码作用于归约，源只取前 8 项。

表 9-1　两条标准 RVV 基线的配置

基线	乘法配置	归约配置	乘积寄存器
RVV-m1	e8、m1、VL=8，目的 EMUL=2	e16、m2、VL=8	对齐的 `v8/v9` 组
RVV-mf2	e8、mf2、VL=8，目的 EMUL=1	e16、m1、VL=8	单个 128 位 `v8`

VL>0 时，基线清零完整目的和种子后计算；VL=0 时跳过运算并保留旧目的。两条路径均在末尾恢复入口 `vl/vtype`。

依据：RISC-V V 1.0 加宽乘法、归约及尾部策略 · 实际汇编源码[材料 13]

2.2 验收内核源码

以下两段逐字摘录实际 `cases.S` 的 vm=0 内核。入口 `a3` 保存原 VL，`a4` 保存原 `vtype`。

RVV-m1：

```
    beqz a3,rvv_m1_vm0_restore
    vsetivli zero,8,e8,m1,tu,mu
    vwmul.vv v8,v1,v2
    vsetivli zero,4,e32,m1,tu,mu
    vmv.v.i v3,0
    vmv.v.i v16,0
    vsetivli zero,8,e16,m2,tu,mu
    vwredsum.vs v3,v8,v16,v0.t
rvv_m1_vm0_restore:
    vsetvl zero,a3,a4
```

RVV-mf2：

```
    beqz a3,rvv_mf2_vm0_restore
    vsetivli zero,8,e8,mf2,tu,mu
    vwmul.vv v8,v1,v2
    vsetivli zero,4,e32,m1,tu,mu
    vmv.v.i v3,0
    vmv.v.i v16,0
    vsetivli zero,8,e16,m1,tu,mu
    vwredsum.vs v3,v8,v16,v0.t
rvv_mf2_vm0_restore:
    vsetvl zero,a3,a4
```

源码中的每一步分别承担以下职责：

1. `beqz` 保留 VL=0 的旧目的；非零路径固定 VL=8，只计算前 8 项。
2. `vwmul.vv` 加宽乘法；两条路径的区别是乘积寄存器组大小及后续归约的 LMUL。
3. e32/m1、VL=4 配置下的两次 `vmv.v.i` 分别清零完整目的和归约种子；随后以 `tu` 保留高位零。
4. `vsetvl` 恢复入口配置。vm=1 内核仅在对应归约中省去 `v0.t`，其余流程相同。

非零路径各含 9 条静态指令：分支、三次 `vsetivli`、乘法、两次清零、归约和配置恢复。VDot 核心段为单条自定义指令；延迟比较使用完整操作的实际周期，另见第 10 章。

2.3 等价矩阵与检查方法

六组数据包括正数、混合符号、正负极值、零掩码及稀疏掩码。源高半部与旧目的填入非零数据，以检查 VL=0 保留旧目的、VL=16 仍只计算低 8 项、高 96 位清零以及源和掩码保持。

表 9-2　等价验证矩阵

维度	取值或数量
数据集	6 组
`vm`	0、1
VL	0、8、16
尾部 / 掩码策略	4 种组合
用例总数	6 × 2 × 3 × 4 = 144
每例路径	VDot、RVV-m1、RVV-mf2
内核调用总数	144 × 3 = 432

Python 整数模型生成独立期望值。C 检查器逐路径核对完整 `v0`～`v3` 和约定 CSR，再比较三条路径；四个向量寄存器共 64 字节（512 位），按 64 位字比较全部内容。状态检查包括 VS=Dirty、`vstart=0`，以及保持不变的 `vcsr/fflags`。

反汇编检查覆盖六个内核范围，确认标准基线段包含加宽乘法与归约且没有 custom-0 指令。负向验证在参考模型 RAM 中扰动期望字节，用于检查结果比较器能否拒绝错误。

2.4 构建与整核运行

等价应用使用 RISC-V 工具链独立编译，随后由固定的修复版 emu 加载，并通过 `--diff` 连接源码 NEMU 参考库。该次服务器运行采用 seed=1，周期上限为 3,000,000，墙钟上限为 1800 秒；实际应用构建用时 3.726 秒，整核运行用时 680.930 秒。主机耗时用于复现安排，指令延迟另由第 10 章的客体周期测量给出。

复现入口：应用源码与构建文件[材料 14]．完整构建及运行命令[材料 15]

3 结果

等价矩阵在本地参考执行和服务器 RTL/NEMU 协同运行中均通过，负向测试按预期拒绝错误结果。 两种验证环境的应用和参考库身份分列保存。

表 9-3　验证结果

验证层	结果	检查内容
本地应用构建	退出码 0	生成等价测试镜像
本地源码 NEMU	矩阵全部通过	完整向量与约定状态，对照独立期望
完整 C `main`	返回 0，PASS	路径间比较和完整矩阵
检查器负向验证	返回 1，按预期拒绝	扰动期望字节后发现差异
服务器整核 DiffTest	等价矩阵通过	显式源码 NEMU、DiffTest、GOOD TRAP
服务器退出状态	构建、运行、脚本、`tee` 均为 0	被测进程与日志收集分别检查

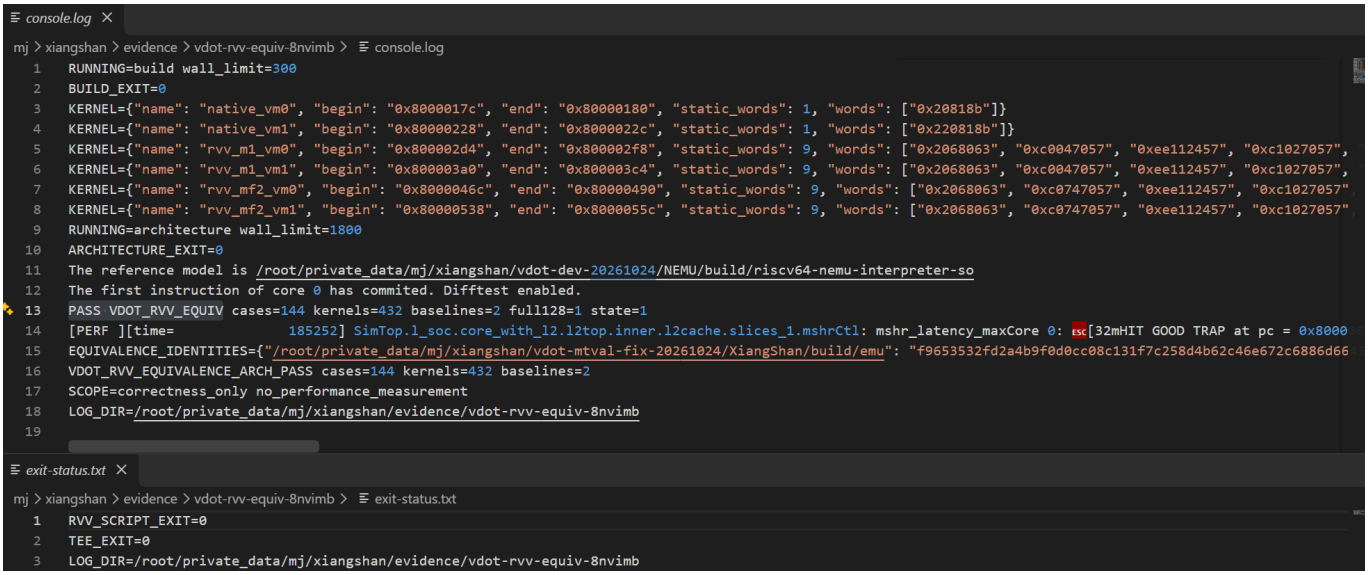


图 9-1　两种 RVV 基线完成表 9-2 的等价矩阵，日志显示 PASS、DiffTest、GOOD TRAP 与零退出码（原证据 S3-F01）。完整日志中的正常结束地址为 `0x80000858`，位于截图右侧裁剪范围之外。

证据：本地结果[材料 16]．服务器整核核验[材料 17]．运行产物与材料索引

4 总结与实验边界

RVV-m1 与 RVV-mf2 均能在固定调用约定下复现 `vdot.vv` 的结果和约定状态，因而可作为第 10 章的两条标准组合基线。配置、清零和恢复开销由各性能接口明确纳入。

- 等价结果覆盖表 9-2 的合法状态矩阵；非法状态异常与源、目的、掩码别名由第 8 章的独立套件检查。
- `v8`/`v9`/`v16` 和 `t0` 为允许破坏的临时寄存器，不承诺任意寄存器布局下的透明替换。
- 等价检查确定正确性；分层性能报告分别给出依赖链延迟、八路批处理和含访存的小数组测量。

分层性能测量与分析

本文要点

- 对应赛题 3.3，完整操作依赖链的增量延迟为 VDot **7 周期/次**、RVV-m1 **55 周期/次**、RVV-mf2 **47 周期/次**；延迟加速比分别为 **7.86 倍、6.71 倍。**
- 八路寄存器批处理的平均增量成本为 **1、6、3 周期/次**；相对两条基线的增量吞吐加速比分别为 **6 倍、3 倍。**
- FU 内部输入握手到完成为 **2 周期**；整核依赖链还计入依赖传递和完整操作接口的成本，两者分别计量。
- 扩展小数组实验表明：加入加载、完整结果存储与末尾排空后，N=128 时 VDot / RVV-mf2 为 **2239 / 2241 周期**，完整操作延迟优势没有等比例转化为整体收益。该结果支持继续考察数据搬运与计算的组织方式。

表 10-1　三类测量与两条 Baseline 的对比。加速比统一为“基线周期成本 / VDot 周期成本”。

测量与指标	VDot	RVV-m1	RVV-mf2	相对 RVV-m1 / RVV-mf2 加速比	指标含义
依赖链：增量延迟	7 周期/次	55 周期/次	47 周期/次	7.86 / 6.71	Latency 加速比；两段长度扫描斜率一致
八路寄存器批处理：增量成本	1 周期/次	6 周期/次	3 周期/次	6 / 3	N=8→128 的平均周期斜率；吞吐指标
八路寄存器批处理：N=128 整窗	216 周期	906 周期	519 周期	4.19 / 2.40	含准备、边界与消费者的整窗比
扩展小数组：N=128 整窗	2239 周期	2249 周期	2241 周期	约 1.00 / 约 1.00	VDot 分别少 10 / 2 周期

1 对象

1.1 实现路径与实验条件

比较对象为自定义指令 `vdot.vv` 的 VDot 实现路径，以及两种已通过等价验证的标准指令组合 RVV-m1、RVV-mf2。实验采用锁定版本的单核 MinimalConfig，VLEN=128，seed=1；RTL 仿真器、NEMU 参考库和 Flash 在各组测量间保持固定。

表 10-2　测量计划与正式样本数量。N 表示一个窗口内的点积操作数。

测量名称	N 或负载	预热调用数	正式样本数	每点正式样本数
计时预检	empty、addi16、addi64、VDot、RVV-m1、RVV-mf2	6	36	6
依赖链测量	N=1、8、32，另设空对照	10	30	3
八路批处理测量	N=8、32、128，另设空对照	10	30	3
小数组测量	N=8、32、128，另设空对照	10	30	3

基础功能、异常和流水套件统一见协同验证报告。本章只统计计时应用的样本，不重复累计功能用例。

证据：运行身份与原件索引 · RVV 等价报告

1.2 术语和记法

- 指令名写作 `vdot.vv`，实现路径写作 VDot；两条标准组合路径写作 RVV-m1、RVV-mf2。
- 周期 C(N) 表示完整计时窗口的客体处理器周期。增量成本为 $[C(N_2)-C(N_1)]/(N_2-N_1)$；增量比比较两条路径的增量成本，整窗比直接比较 C(N)。
- 整数周期保留原值；派生斜率和比值通常保留两位小数。接近 1 的比值同时列出周期差，避免舍入掩盖差异。
- 源码和日志中的 `native` 对应 VDot，`rvv_m1/rvv_batch_m1` 对应 RVV-m1，`rvv_mf2/rvv_batch_mf2` 对应 RVV-mf2；原始变量名保留在截图中。

2 方法

2.1 计时窗口

计时窗口使用显式边界与结果依赖，使结束读数位于目标计算之后。锁定的香山实现会放宽部分只读 CSR 访问的调度次序；因此测量程序用写 `mscratch` 的 `CSRRW` 保留等待前序、阻止后序的控制，并在负载之后消费结果。

```
外侧边界 → C0 → 边界 → I0 → 边界 → 负载 → 结果消费者
         → 边界 → I1 → 边界 → C1 → 外侧边界 → 保存与恢复
```

上述示意与实际汇编对应如下：

1. C0/C1 读取 `mcycle`，I0/I1 读取 `minstret`。
2. 预检消费者通过 `vmv.x.s` 依赖结果寄存器 v3，并检查低字节为 36；其他测量按各自输出设置消费者。
3. 原始周期差保留四个内部写边界、两个退休计数读取和消费者的成本；空负载用于观察公共开销。
4. `mscratch` 的入口值保存在 t6，函数返回前恢复并检查。

这套边界依据锁定微架构的调度行为建立。小数组的存储排序另由 `fence rw,rw` 处理；两者的测量含义见第 4 节。

证据：计时汇编[材料 18] · RISC-V 计数器规范

2.2 三类负载的组织

表 10-3　计时窗口中的工作与数据依赖。

测量	输入与依赖	窗口内的工作	窗口外的工作
依赖链测量	每次读写 v1，后一操作读取前一结果	展开的点积序列；每次 RVV 配置、清零和恢复；最终结果消费	输入装载、结果存储、自检和打印
八路批处理测量	v1/v2 为固定输入，v24～v31 轮转保存结果	窗口起始清零、窗口结束恢复；每 8 次共享乘法与归约配置；读取并汇总 8 个结果	输入装载、结果存储、自检和打印
小数组测量	逐项使用不同点积输入，同一小数组预热后重复	地址计算、加载、配置、点积、完整 128 位输出存储、末尾排空和读回	初始化、输出哨兵填充、自检和打印

八路批处理的结果不回授到输入源，但目的复用之间仍存在既有 oldVd 依赖。两种 RVV 路径的配置复用、清零和恢复成本均计入相应窗口。

小数组每项读取两组各 8 个有符号字节，保存 16 字节输出。起点前排空初始化存储，终点前执行 `fence rw,rw` 并读回末项结果；应用同时检查全部输入、完整输出、保护区及指定状态。

证据：依赖链源码[材料 19]．八路批处理源码[材料 20]．小数组源码[材料 21]

3 结果

3.1 计时预检

六种负载的退休指令增量均符合预期；每种负载的正式样本周期一致。空负载为 74 周期，VDot 单操作为 81 周期，说明单次窗口中公共开销占比较大。后续采用不同 N 的批量窗口计算增量成本。

表 10-4　计时预检结果；周期列的最小值、中位数和最大值相同。

负载	负载指令数	窗口周期	退休指令增量
empty	0	74	4
addi16	16	86	20
addi64	64	134	68
VDot	1	81	5
RVV-m1	9	130	13
RVV-mf2	9	122	13

预期退休增量为负载指令数加 4：起始退休计数读取、两道内部写边界以及结果消费者。结束计数读取自身尚未计入所读值。

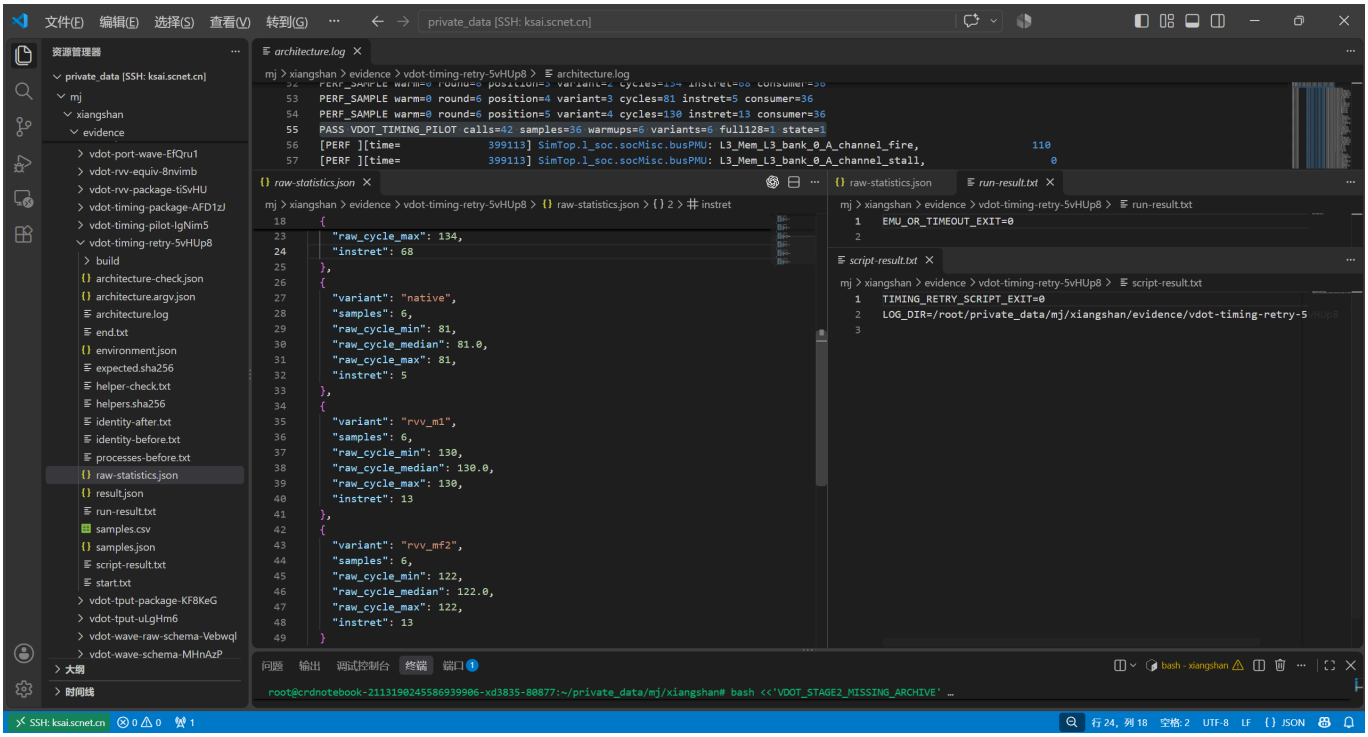


图 10-1　计时预检通过记录及运行、脚本退出状态；完整六组校准读数见图 10-2（原证据编号 S3-F02a）。

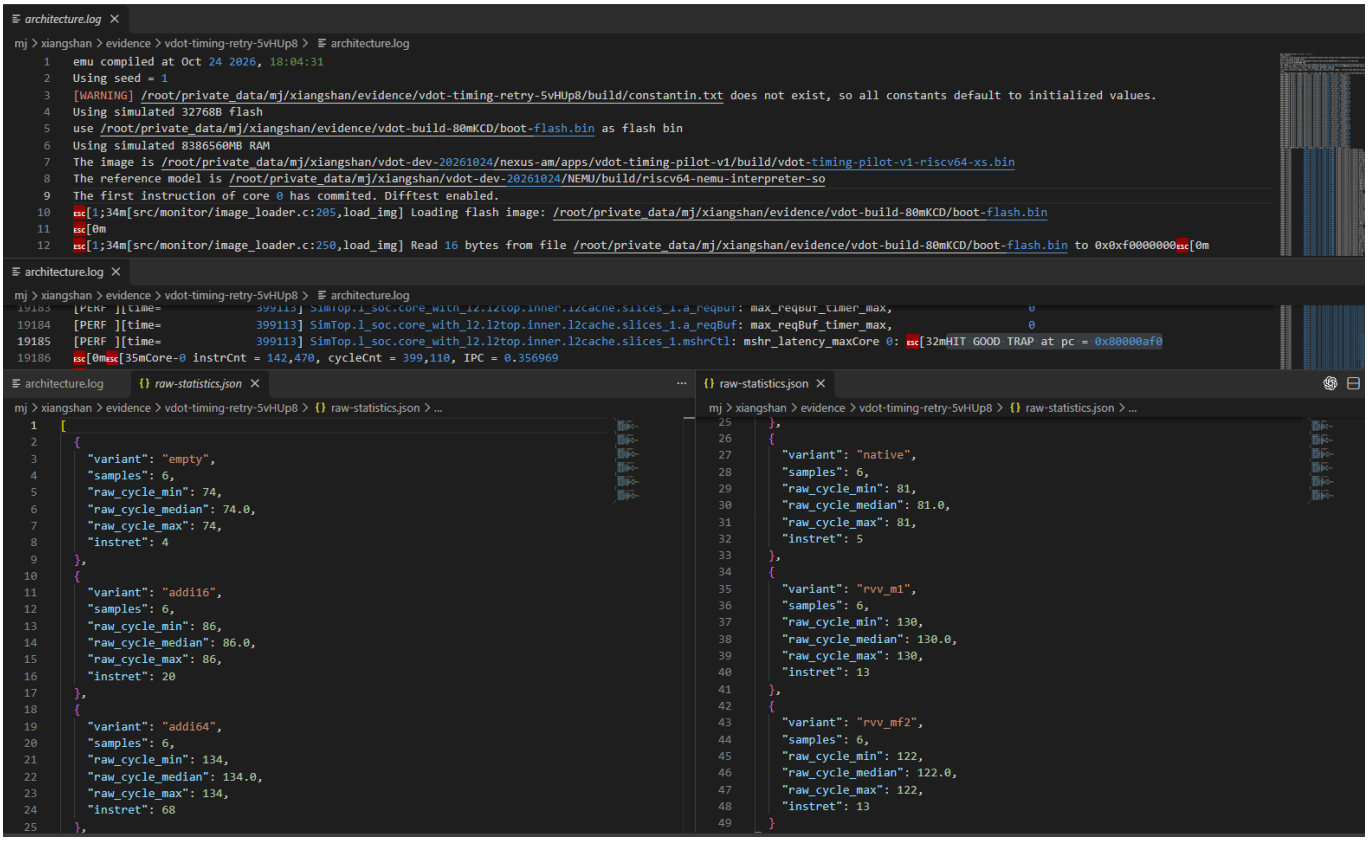


图 10-2　预检完整校准结果：empty 为 74 周期，VDot 为 81 周期，RVV-m1 和 RVV-mf2 分别为 130、122 周期（原证据编号 S3-F02b）。

证据：预检原始统计[材料 22]．预检运行记录[材料 23]

3.2 核心结果：完整操作的 Latency 加速比

VDot 每新增一次完整操作为 **7 周期**，RVV-m1 为 **55 周期**，RVV-mf2 为 **47 周期**。以基线斜率除以 VDot 斜率，得到 **7.86 倍、6.71 倍的 Latency 加速比**；两段长度扫描得到相同结果。

表 10-5　依赖链周期、退休指令数及增量延迟。每个测量单元格依次列出“周期 / 退休条数”，每点最小值、中位数和最大值相同。

实现路径	N=1：周期 / 条	N=8：周期 / 条	N=32：周期 / 条	增量：1→8（周期/次）	增量：8→32（周期/次）
VDot	81 / 5	130 / 12	298 / 36	7	7
RVV-m1	130 / 13	515 / 76	1835 / 292	55	55
RVV-mf2	122 / 13	451 / 76	1579 / 292	47	47

三个测量点分别满足 `C(N)=74+7N`、`75+55N` 和 `75+47N`。RVV 的截距与空对照 74 周期相差 1 周期，因此使用相邻点差分计算增量成本，消去各路径固定截距。这一斜率表示完整操作在整核依赖链中的增量延迟；VDot FU 内部由 `CertainLatency(2)` 与握手波形验证为 2 周期，另见数据通路设计[材料 24]。

空对照为 74 周期、4 条退休增量；表 10-5 的退休计数包含窗口内的配置、结果消费及边界指令。N=32 时，RVV-mf2 / VDot 的整窗比为 `1579/298 ≈ 5.30`。增量比 6.71 与整窗比 5.30 分别回答周期增长和完整窗口成本的问题。

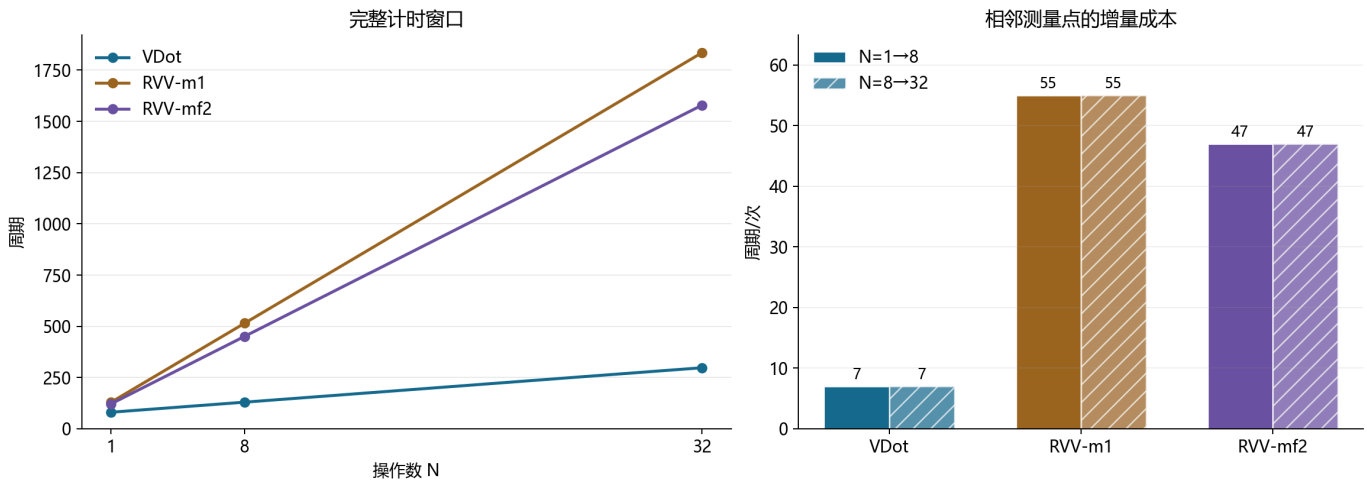


图 10-3　依赖链窗口周期与增量成本，VDot 两段均为 7 周期/次，RVV-mf2 两段均为 47 周期/次（原证据编号 S3-F03b，依据原统计重绘）。

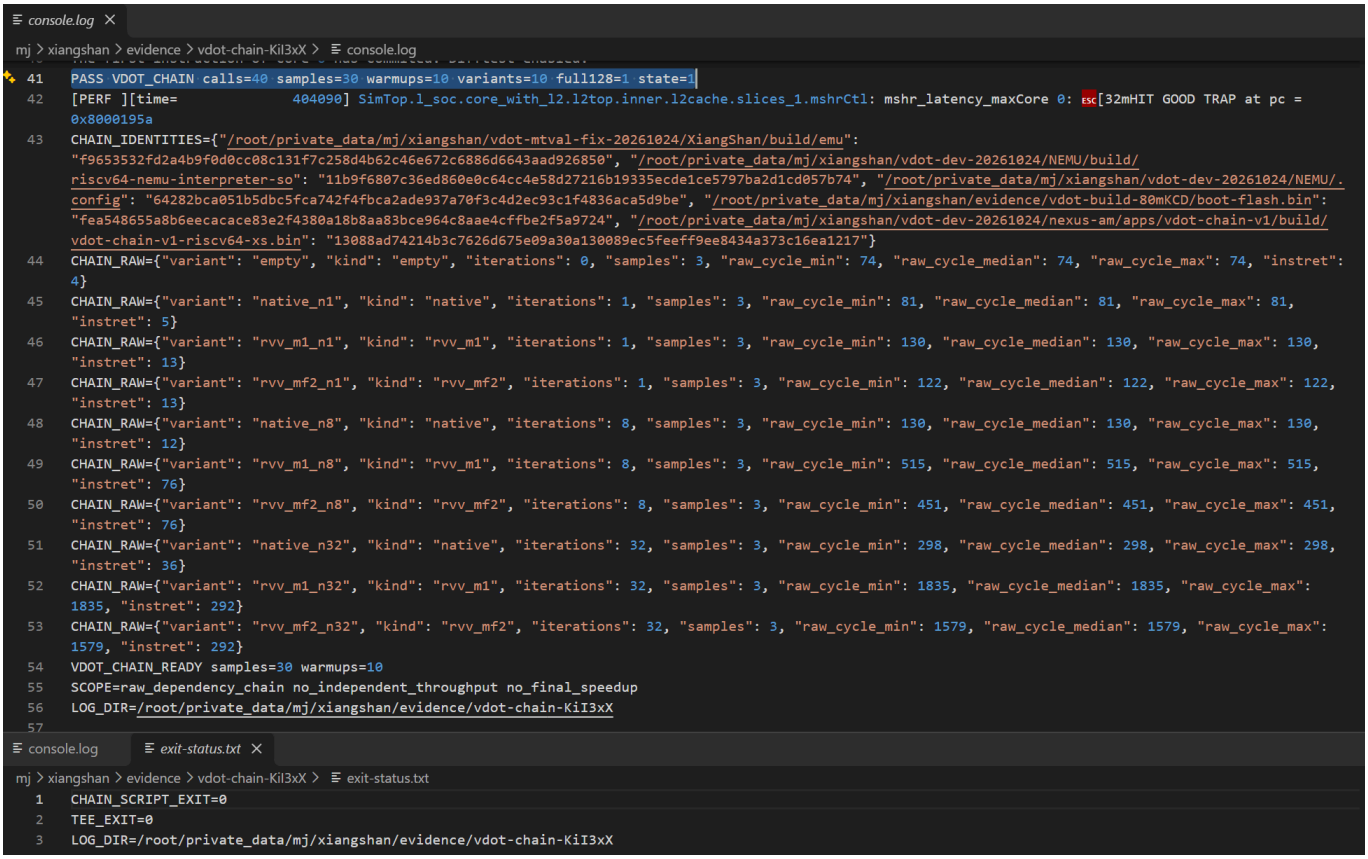


图 10-4　依赖链原始结果记录，包含各长度的周期、退休指令增量及结束状态（原证据编号 S3-F03a）。

证据：依赖链原始统计[材料 25]，依赖链运行记录[材料 26]

3.3 补充结果：八路寄存器批处理的吞吐加速比

八路批处理中，VDot 的两段增量均为 **1 周期/次**，RVV-m1 为 **6 周期/次**，RVV-mf2 的全跨度平均值为 **3 周期/次**；相对两条基线的增量吞吐加速比为 **6 倍、3 倍**。N=128 时的整窗比分别为 **4.19、2.40**。

表 10-6　八路批处理周期、退休指令数及增量成本。单元格为“周期 / 退休条数”，每点最小值、中位数和最大值相同。

实现路径	N=8：周期 / 条	N=32：周期 / 条	N=128：周期 / 条	增量：8→32（周期/次）	增量：32→128（周期/次）
VDot	96 / 26	120 / 50	216 / 146	1	1
RVV-m1	186 / 47	330 / 101	906 / 317	6	6
RVV-mf2	159 / 47	232 / 101	519 / 317	3.04	2.99

批处理空对照为 88 周期、18 条退休增量。VDot 的三个点满足 $C(N)=88+N$，RVV-m1 满足 $C(N)=138+6N$。RVV-mf2 在 N=8→128 的平均增量为 3 周期/次，N=32 的测量点比两端连线高 1 周期，表中保留两段差分。

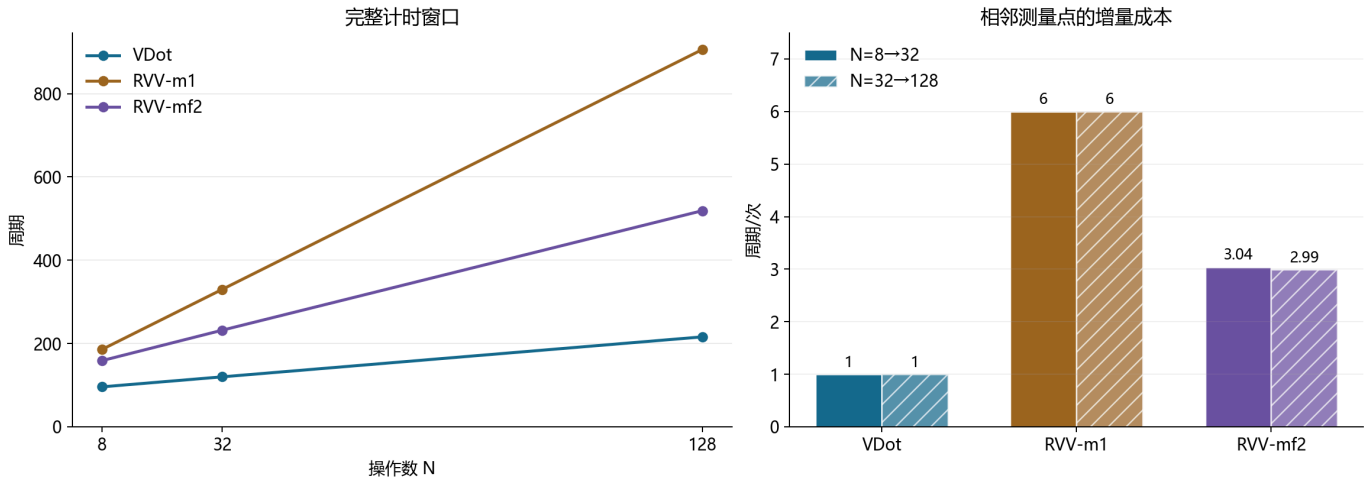


图 10-5　八路批处理窗口周期与增量成本，RVV-mf2 的两段斜率为 3.04 和 2.99 周期/次（原证据编号 S3-F04b，依据原统计重绘）。

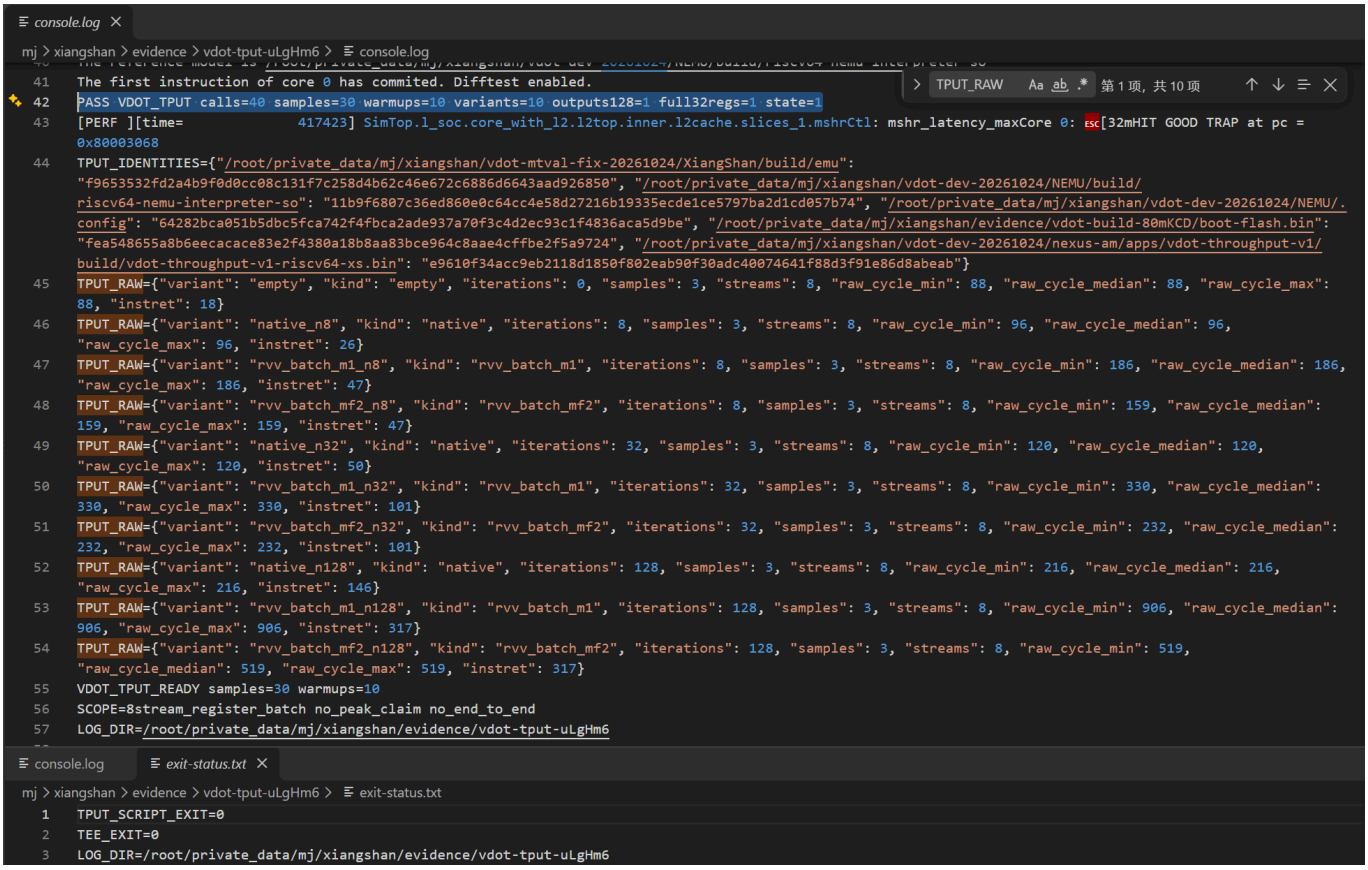


图 10-6　八路批处理原始结果记录，N=128 时 VDot 为 216 周期，RVV-mf2 为 519 周期（原证据编号 S3-F04a）。

证据：八路批处理原始统计[材料 27]，八路批处理运行记录[材料 28]

3.4 扩展探究：细粒度点积的数据搬运开销与算存比边界

同一算术操作的加速收益取决于外层数据组织。 加入加载、完整结果存储和末尾排空后，N=128 时 VDot 为 **2239 周期**，RVV-mf2 为 **2241 周期**，相差 **2 周期**。这个结果把寄存器内的完整操作优势与包含数据搬运的整体成本区分开来。

小数组每项加载两组各 8 字节输入，并生成 16 字节输出；程序每 8 项计算后集中存储 8 个结果，在整个窗口末尾执行一次 `fence rw,rw` 和末项读回。它没有在每次点积后执行 fence。N=128 的 VDot 整窗平均成本约为 **17.49 周期/次**，包含配置、地址计算、加载、存储和公共边界。

表 10-7　小数组周期、退休指令数和增量成本。周期取中位数，括号为有波动时的最小～最大值；其余测量点三值相同，斜率单位为周期/次。

实现路径	N	周期	退休条数	相对上一 N 的周期增量/次
VDot	8	287	65	—
VDot	32	674（674～676）	239	16.13
VDot	128	2239（2239～2240）	935	16.30
RVV-m1	8	320	84	—
RVV-m1	32	705	285	16.04
RVV-m1	128	2249	1089	16.08
RVV-mf2	8	312	84	—
RVV-mf2	32	698（696～698）	285	16.08
RVV-mf2	128	2241	1089	16.07

小数组空对照为 100 周期、6 条退休增量。N=8、32、128 时，RVV-mf2 / VDot 的整窗比分别约为 1.09、1.04、1.00，对应周期差为 25、24、2。N=128 时，VDot 比 RVV-mf2 少退休 154 条指令，但窗口周期仅减少约 0.09%。

硬件计数同时显示了加载、存储和排空活动，见表 10-8。其中请求有效计数 `sbuffer_req_valid` 与接受请求计数 `sbuffer_req_fire` 有明显差异。这些计数覆盖整个应用，包括预热、初始化、自检和打印；等待项也可能重叠，因此用来判断后续应检查哪些资源，而非直接换算单个内核的停顿占比。

表 10-8　小数组应用的全程硬件计数，日志时间字段为 time=460257。

来源	字段	值
ROB	clock_cycle	460258
ROB	commitInstr	210343
ROB	waitLoadCycle	182322
ROB	waitStoreCycle	161379
ROB	waitFenCycle	860
Sbuffer	sbuffer_req_valid	31229
Sbuffer	sbuffer_req_fire	16772
Sbuffer	dcache_req_valid	3910
Sbuffer	dcache_req_fire	3903

计数来源：完整计数器路径与读数[材料 29]

从 Amdahl 定律的角度，只有被缩短的点积操作及其配置序列能直接贡献整体加速；外层工作及其与计算的重叠方式决定收益如何传递。当前结果与“数据搬运和排空开销限制收益”的解释相容，适合作为优化假设；上述全程计数尚不能将 VLSU、Sbuffer 或其他资源单独确定为主导瓶颈。

由此提出的下一组实验采用分段点积：每 8 次或 16 次点积输出一个累加结果，并为三条路径定义相同的累计位宽和输出。寄存器累加与中间结果落存两种组织方式构成受控对照，用于检验减少数据搬运能否改善整体性能。

> [TODO / 待后续补齐] 分段点积应用已通过本地参考模型预检，服务器 RTL 性能尚未回传；本章保留现有测量结果，待同语义对照完成后补充分析。

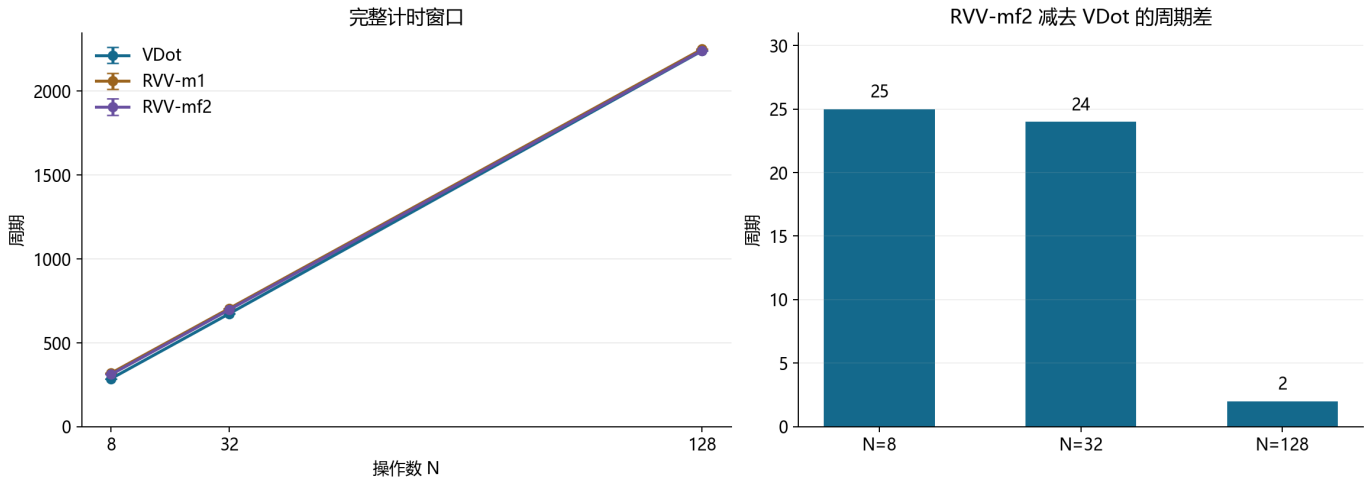


图 10-7　小数组周期中位数与最小～最大范围，右图直接列出 RVV-mf2 与 VDot 的周期差（原证据编号 S3-F05b，依据原统计重绘）。

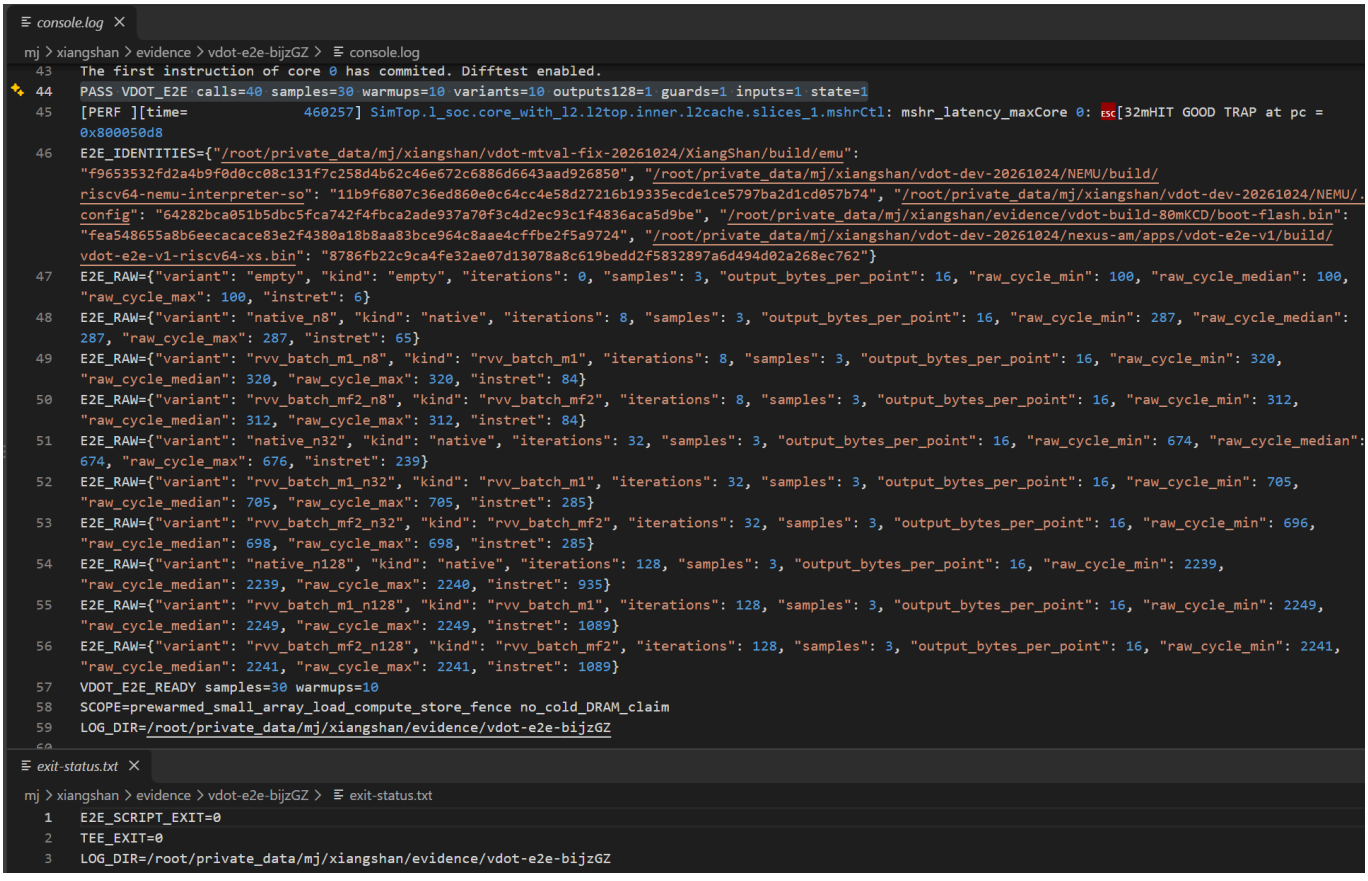


图 10-8　小数组原始统计与应用检查结果，N=128 时 VDot 中位数为 2239 周期、RVV-mf2 为 2241 周期（原证据编号 S3-F05a）。

证据：小数组原始统计[材料 30]．完整逐轮样本[材料 31]．运行记录[材料 32]

4 总结与实验边界

完整操作依赖链给出相对 RVV-m1 / RVV-mf2 的 **7.86 倍 / 6.71 倍延迟加速**，直接回应赛题 3.3。八路批处理进一步给出 **6 倍 / 3 倍增量吞吐加速**，说明配置复用与目的轮转对周期成本的影响。扩展小数组则说明，完整数据通路的收益需要结合加载、结果存储和排空成本评价。

- **实验配置**：固定版本、单核 MinimalConfig、VLEN=128、seed=1，样本见表 10-2；依赖链使用固定正数输入，RVV 每次配置、清零和恢复均计时。
- **指标口径**：7 周期是整核依赖链增量延迟，2 周期是 FU 内部握手间隔；吞吐斜率和整窗比各有用途，不合并成单一倍数。批处理采用八路目的轮转，保留 oldVd 依赖。
- **测量边界**：CSRRW 按锁定实现约束调度，fence 单独处理内存顺序。小数组为预热后重复使用的负载；误差范围表示实测最小～最大值。
- **因果分析**：计数器覆盖全程且等待项可能重叠；具体资源归因和降低存储频率的收益留给同语义、分窗口的受控对照。

第三阶段复现环境、产物哈希与材料路径索引

正文包含构建方法、核心实现与实验结果。本附录用于核对版本、识别运行产物和定位完整工程材料。

1 复现环境与源码版本

表 S3-A1　环境、配置与源码版本。

对象	配置或版本
平台与工具	SCNet Linux 服务器；JDK 21、Mill 0.12.3、Verilator 5.052；应用使用服务器 RISC-V GNU 工具链
整核配置	单核 MinimalConfig、VLEN=128、EMU_TRACE=1；性能运行 seed=1
XiangShan 基版本	`16ae9ddcda54fc9a2fddffad73174cf793ac7814`，叠加完整 VDot 与 mtval 修复补丁
NEMU 基版本	`a48a11f208ff81ad765591d83271d1b339fb3861`，叠加 vdot.vv 参考语义
状态交换	固定配置 regcpy 源码布局为 1400 字节，向量区为 512 字节；方法见第 8 章
应用接口	等价验证与三类性能负载分别编译，计时窗口和样本数见第 10 章

2 关键产物哈希

以下为实验记录中保存的 SHA256，配合第 3 节的完整清单使用。

表 S3-A2　关键产物 SHA256。

产物	SHA256
修复版 emu	`f9653532fd2a4b9f0d0cc08c131f7c258d4b62c46e672c6886d6643aad926850`
服务器 NEMU 参考库	`11b9f6807c36ed860e0c64cc4e58d27216b19335ecde1ce5797ba2d1cd057b74`
服务器 NEMU 配置	`64282bca051b5dbc5fca742f4fbca2ade937a70f3c4d2ec93c1f4836aca5d9be`
共享 Flash	`fea548655a8b6eecacace83e2f4380a18b8aa83bce964c8aae4cffbe2f5a9724`
本地 API / RVV 检查用 NEMU	`5ba6caf77513904c6ea541c4c145720a6bebe63f983eefa5b73e53441069c872`
服务器 RVV 等价应用 BIN	`bf29b9367a4b9045b139224bb5b2b6502bdeae005969abdc6129ba0fd6c76010`
计时预检应用 BIN	`8545abf1afb71ec405a8c81bada3fe5054a332964436eb7976a824edaf91bda6`
依赖链应用 BIN	`13088ad74214b3c7626d675e09a30a130089ec5feeff9ee8434a373c16ea1217`
八路批处理应用 BIN	`e9610f34acc9eb2118d1850f802eab90f30adc40074641f88d3f91e86d8abeab`
小数组应用 BIN	`8786fb22c9ca4fe32ae07d13078a8c619bedd2f5832897a6d494d02a268ec762`

3 工程材料路径索引

所有路径相对于配套项目根目录。材料名称说明其用途；目录中的随机后缀只用于区分服务器保存的实验批次。

表 S3-A3　工程材料与保存路径。

材料号	内容	项目相对路径
1	语义合同	docs/vdot-spec-v0.1.md
2	NEMU 源码快照	artifacts/vdot-integration-2026-10-24/delivery/source-snapshot/NEMU
3	译码源文件	artifacts/vdot-integration-2026-10-24/delivery/source-snapshot/NEMU/src/isa/riscv64/instr/decode.c
4	语义源文件	artifacts/vdot-integration-2026-10-24/delivery/source-snapshot/NEMU/src/isa/riscv64/instr/rvv/vdot.h
5	构建与复现入口	reports/stage3/delivery.md
6	接口布局核对	artifacts/scnet-abi-2026-10-24/abi-review.json
7	API 原始结果	artifacts/vdot-integration-2026-10-24/nemu-api-151012/result.json
8	异常修复回归	artifacts/vdot-mtval-fix-2026-10-24/server-xYGYlZ/review.json
9	流水套件回归	artifacts/vdot-pipeline-2026-10-24/server-JuGUz6/review.json
10	差分诊断与 ELF 映射	artifacts/vdot-matrix-2026-10-24/server-OMcrpf/diagnosis.md
11	修复设计与补丁	artifacts/vdot-mtval-fix-2026-10-24/README.md
12	代表性微架构波形	docs_final/03_阶段二_指令设计与RTL实现/功能测试与微架构时序验证.md
13	实际汇编源码	benchmarks/rvv-baseline/vdot-rvv-equiv-v1/cases.S
14	应用源码与构建文件	benchmarks/rvv-baseline/vdot-rvv-equiv-v1
15	完整构建及运行命令	artifacts/vdot-rvv-equivalence-2026-10-26/server-paste-command.txt
16	本地结果	artifacts/vdot-rvv-equivalence-2026-10-26/local-170237/result.json
17	服务器整核核验	artifacts/vdot-rvv-equivalence-2026-10-26/server-8nvimb/review.json
18	计时汇编	benchmarks/rvv-baseline/vdot-timing-pilot-v1/cases.S
19	依赖链源码	benchmarks/rvv-baseline/vdot-chain-v1/cases.S
20	八路批处理源码	benchmarks/rvv-baseline/vdot-throughput-v1/cases.S
21	小数组源码	benchmarks/rvv-baseline/vdot-e2e-v1/cases.S
22	预检原始统计	artifacts/vdot-timing-pilot-2026-10-26/server-5vHUp8/raw-statistics.json
23	预检运行记录	artifacts/vdot-timing-pilot-2026-10-26/server-5vHUp8/review.json
24	数据通路设计	docs_final/03_阶段二_指令设计与RTL实现/vdot微架构与数据通路设计.md
25	依赖链原始统计	artifacts/vdot-chain-2026-10-26/server-KiI3xX/raw-statistics.json
26	依赖链运行记录	artifacts/vdot-chain-2026-10-26/server-KiI3xX/review.json
27	八路批处理原始统计	artifacts/vdot-throughput-2026-10-26/server-uLgHm6/raw-statistics.json

材料号	内容	项目相对路径
28	八路批处理运行记录	artifacts/vdot-throughput-2026-10-26/server-uLgHm6/review.json
29	完整计数器路径与读数	artifacts/stage3-closeout-2026-10-27/e2e-read/perf-context.json
30	小数组原始统计	artifacts/vdot-e2e-2026-10-27/server-bijzGZ/raw-statistics.json
31	完整逐轮样本	artifacts/stage3-remote-archive-2026-10-27/server-MIwNSR/complete-samples.md
32	运行记录	artifacts/vdot-e2e-2026-10-27/server-bijzGZ/review.json
33	RVV 等价应用身份	artifacts/vdot-rvv-equivalence-2026-10-26/server-8nvimb/identities.json
34	计时运行产物身份	artifacts/vdot-timing-pilot-2026-10-26/server-5vHUp8/identities.json
35	完整日志、样本及文件清单	artifacts/stage3-remote-archive-2026-10-27/server-MIwNSR
36	性能宿主耗时与计数原记录	reports/stage3/performance-evidence.md